

Project 02: Operating System Performance

Sharie Rhea

Southern New Hampshire University

CS 510: Operating Systems Principles and Applications

Brian Enochson

April 5, 2026

Algorithmic optimizations within operating systems (OSes) are evaluated using scheduling metrics and handling of failure states. Scheduling algorithms aim to maximize central processing unit (CPU) utilization by optimizing the throughput (Silberschatz et al., 2023). This in turn involves minimizing the turnaround time, which is the time from the submission of a process to its completion, including time spent waiting, executing, and performing input/output (I/O) operations (Silberschatz et al., 2023). However, for some interactive processes, measuring the time from process submission to the first executing period—the response time—may be a better metric (Silberschatz et al., 2023). In multicore systems, concurrency improves performance but introduces risks. Deadlock, where each process is waiting for an event that can only be caused by another process in the set, is a failure state where some of the system's resources become locked until manual intervention is performed (Silberschatz et al., 2023). Atomic hardware instructions can be used to ensure only one critical section is performed at a time, guaranteeing memory and resource safety and preventing undefined states (Silberschatz et al., 2023).

Key OS performance metrics include CPU and memory utilization. Under stress, it is desirable to use the CPU as efficiently as possible. However, sustained high usage with few processes may indicate insufficient hardware (Silberschatz et al., 2023). Similarly, reaching maximum random access memory (RAM) usage is a critical concern. Once RAM is exhausted, excessive paging can lead to thrashing and severe performance degradation. In this scenario, some OSes will choose to forcefully terminate memory-intensive processes in order to maintain system responsiveness.

Virtual memory is one method for optimizing memory allocation through the abstraction of physical memory space in logical pages. This allows processes to view memory as one contiguous block, which may even be larger than the amount of physical memory available on

the system (Silberschatz et al., 2023). The OS uses page allocation tables to map virtual pages to physical memory, and loads sections into RAM as needed through demand paging (Silberschatz et al., 2023). When RAM becomes full, old or stale pages must be swapped out to make room for new data. The enhanced second-chance page replacement algorithm utilizes read and write bits, sometimes called reference and dirty bits, to track pages that have been read and or modified recently (Silberschatz et al., 2023). The four possible combinations allows for the OS to prioritize pages that have not been modified, as they do not need to be written out to disk and are thus faster to swap, and pages that are unlikely to be needed in the near future (Silberschatz et al., 2023).

Virtual memory also improves security and stability by isolating processes and preventing unauthorized memory access. Kernel-mode protections further enforce this separation, ensuring processes cannot interfere with one another (Silberschatz et al., 2023).

Resource allocation of the CPU is optimized through scheduling algorithms such as multilevel feedback queue, which dynamically adjust processes into priority queues to balance responsiveness (Silberschatz et al., 2023). This ensures that each process is able to execute in a reasonable amount of time according to its priority: real-time, system, interactive, or batch (Silberschatz et al., 2023). Multiprocessing systems also utilize push and pull migration techniques to ensure that no core remains idle while there is still work to be done. However, cache coherency and non-uniform memory architecture (NUMA) must be taken into account to ensure that the overhead incurred does not negate the possible performance improvements (Silberschatz et al., 2023). I/O performance is improved through buffering and caching, reducing the overhead of slow device operations (Silberschatz et al., 2023). This also ensures device compatibility and stability through a uniform interface.

References

Silberschatz, A., Galvin, P. B., & Gagne, G. (2023). *Operating System Concepts* (10th ed.).

Wiley. <https://learn.zybooks.com/zybook/CS-510-10435.202617-1>